

 CNN_dogs_vs_cats_COLAB_ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🗨️ ⚙️

Share

Gemini



🔍 Commands + Code + Text

RAM Disk

✓

⬆️ ⬇️ 🔗 💬 ✎️ 📄 🗑️ ⋮

☰

🔍

⏏️

📁

⌵ Cats_Vs_Dogs Convolution Neural Network

⬆️ ⬇️ 🔗 💬 ✎️ 📄 🗑️ ⋮

Problem Statement:

The task at hand involves classifying images of cats and dogs using a Convolutional Neural Network (CNN).

We have a dataset containing images of cats and dogs and the objective is to train a CNN model to accurately predict the class of each image.

Objectives:

Data Preparation:

- Download and extract the cats vs. dogs dataset.
- Organize the dataset into a structured directory format suitable for TensorFlow's ImageDataGenerator.
- Split the dataset into training and testing sets.

Data Augmentation and Preprocessing:

- Implement data augmentation techniques to increase the diversity of the training dataset, aiding in the model's ability to generalize.

Model Building:

- Normalize the pixel values of the images for optimal model performance.
- Construct a Convolutional Neural Network using TensorFlow and Keras.
- The model should contain multiple convolutional layers, pooling layers, and fully connected layers.

Training:

- Compile the model and train it on the prepared dataset.
- Utilize categorical cross entropy as the loss function and stochastic gradient descent as the optimizer.
- Train the model for a sufficient number of epochs to achieve good performance.

Evaluation:

- Evaluate the model's performance on the validation set during training to monitor for overfitting.
- After training, assess the model's accuracy and make predictions on the test set.

Prediction:

- Implement a system to make predictions on new images, categorizing them as either cat or dog

The system should be able to take an image (or a batch of images), preprocess it, and pass it through the model for predictions.

✓ 3s [23] ! pip install Pillow

🔄 Requirement already satisfied: Pillow in /usr/local/lib/python3.11/dist-packages (11.2.1)

✓ 6s [5]

```
import numpy as np
import pandas as pd
import os
import zipfile

# for visualization
import matplotlib.pyplot as plt
import matplotlib.image as imread
from PIL import Image
import seaborn as sns

# tensorflow
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img
from tensorflow.keras.utils import image_dataset_from_directory
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping
```

✓ 16s [6]

```
from zipfile import ZipFile

data_path = '/content/drive/MyDrive/dogs_vs_cats.zip' # dataset zip uploaded in google drive

with ZipFile(data_path, 'r') as zip:
    zip.extractall()
    print('The data set has been extracted.')
```

🔄 The data set has been extracted.

✓ 0s [7]

```
path = 'dogs_vs_cats'
classes = os.listdir(path)
classes
```

🔄 ['test', 'train']

⌵ creating directories and dataframe

✓ 0s [8]

```
# directory path
train_dog_dir = 'dogs_vs_cats/train/dogs'
train_cat_dir = 'dogs_vs_cats/train/cats'
test_dog_dir = 'dogs_vs_cats/test/dogs'
test_cat_dir = 'dogs_vs_cats/test/cats'
```

	dog_train	cat_train
0	dog.1875.jpg	cat.11904.jpg
1	dog.5902.jpg	cat.7702.jpg
2	dog.5962.jpg	cat.5917.jpg
3	dog.4842.jpg	cat.4408.jpg
4	dog.7688.jpg	cat.11246.jpg

Visualization of Dogs

Label : Dog



A black dog with white markings on its chest and paws, sitting on a patterned surface.

Label : Dog



A black dog standing on a ground covered with dry leaves.



Penny March 31 2007

```
On [10]: plt.figure(figsize=(10,8))
for i in range(8):
    plt.subplot(2,4,i+1)
    image_path = 'dogs_vs_cats' + '/train' + '/cats/' + str(data['cat_train'][i])
    img = Image.open(image_path)
    plt.imshow(img)
    plt.title(f"Label : Cat")
    plt.axis('off')
plt.tight_layout()
plt.show()
```





Label : Cat

Label : Cat

Label : Cat

Label : Cat

Normalizing and Data Augmentation through ImageDataGenerator

```
[11] train_datagen = ImageDataGenerator(rescale=1./255,
    rotation_range = 15,
    horizontal_flip = True,
    zoom_range = 0.2,
    shear_range = 0.1,
    fill_mode = 'reflect',
    width_shift_range = 0.1,
    height_shift_range = 0.1)

test_datagen = ImageDataGenerator(rescale=1./255)
```

Splitting the data into Training_set and Testing_set

```
[12] # creating variables for directory of train and test
train_dir = 'dogs_vs_cats/train'
valid_dir = 'dogs_vs_cats/test'

training_set = train_datagen.flow_from_directory(directory=train_dir,
    batch_size=32,
    target_size=(150,150),
    class_mode='binary')

testing_set = test_datagen.flow_from_directory(directory=valid_dir,
    batch_size=32,
    shuffle=False,
    target_size=(150,150),
    class_mode='binary')
```

Found 20000 images belonging to 2 classes.
Found 5000 images belonging to 2 classes.

Model Building

```
[13] model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150,150,3)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2, 2),

    # Flatten Layer
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.BatchNormalization(),

    # Output Layer
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an "input_shape"/"input_dim" argument to a layer. When using Sequential super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
[14] model = Sequential()
model.add(tf.keras.layers.Conv2D(32,(3,3),activation='relu',input_shape=(150,150,3)))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D(2,2))
model.add(tf.keras.layers.Conv2D(64,(3,3),activation='relu'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D(2,2))
```

```

model.add(tf.keras.layers.MaxPooling2D(2,2))
model.add(tf.keras.layers.Conv2D(64,(3,3),activation='relu'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D(2,2))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128,activation='relu'))
model.add(tf.keras.layers.Dropout(0.20))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Dense(1,activation='sigmoid'))

```

▼ model compile

```

[15] model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

```

▼ callbacks

```

[16] learning_rate = ReduceLROnPlateau(monitor='val_accuracy',patience=2,
    factor=0.50, min_lr=0.00001,
    verbose=1)
early_stopping = EarlyStopping(monitor='val_loss',patience=5,
    restore_best_weights=True, verbose=0)

```

▼ Model Fit

```

[17] history = model.fit(training_set,
    epochs=10,
    batch_size=32,
    validation_data=testing_set,
    callbacks=[early_stopping,learning_rate])

```

```

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its
self.warn_if_super_not_called()
Epoch 1/10
625/625 ━━━━━━━━━━━ 162s 245ms/step - accuracy: 0.6337 - loss: 0.7267 - val_accuracy: 0.7272 - val_loss: 0.5362 - learning_rate: 0.0010
Epoch 2/10
625/625 ━━━━━━━━━━━ 148s 237ms/step - accuracy: 0.7494 - loss: 0.5115 - val_accuracy: 0.7870 - val_loss: 0.4483 - learning_rate: 0.0010
Epoch 3/10
625/625 ━━━━━━━━━━━ 149s 239ms/step - accuracy: 0.7910 - loss: 0.4487 - val_accuracy: 0.8046 - val_loss: 0.4275 - learning_rate: 0.0010
Epoch 4/10
625/625 ━━━━━━━━━━━ 151s 242ms/step - accuracy: 0.8135 - loss: 0.4037 - val_accuracy: 0.8214 - val_loss: 0.3924 - learning_rate: 0.0010
Epoch 5/10
625/625 ━━━━━━━━━━━ 155s 248ms/step - accuracy: 0.8415 - loss: 0.3591 - val_accuracy: 0.8276 - val_loss: 0.3954 - learning_rate: 0.0010
Epoch 6/10
625/625 ━━━━━━━━━━━ 152s 243ms/step - accuracy: 0.8581 - loss: 0.3307 - val_accuracy: 0.8138 - val_loss: 0.4474 - learning_rate: 0.0010
Epoch 7/10
625/625 ━━━━━━━━━━━ 0s 228ms/step - accuracy: 0.8719 - loss: 0.2975
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
625/625 ━━━━━━━━━━━ 153s 244ms/step - accuracy: 0.8719 - loss: 0.2975 - val_accuracy: 0.6356 - val_loss: 1.0489 - learning_rate: 0.0010
Epoch 8/10
625/625 ━━━━━━━━━━━ 152s 244ms/step - accuracy: 0.8847 - loss: 0.2810 - val_accuracy: 0.8950 - val_loss: 0.2454 - learning_rate: 5.0000e-04
Epoch 9/10
625/625 ━━━━━━━━━━━ 150s 240ms/step - accuracy: 0.8982 - loss: 0.2451 - val_accuracy: 0.8904 - val_loss: 0.2635 - learning_rate: 5.0000e-04
Epoch 10/10
625/625 ━━━━━━━━━━━ 0s 242ms/step - accuracy: 0.8974 - loss: 0.2432
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
625/625 ━━━━━━━━━━━ 160s 256ms/step - accuracy: 0.8974 - loss: 0.2432 - val_accuracy: 0.8846 - val_loss: 0.2834 - learning_rate: 5.0000e-04

```

▼ Saving the model

```

[18] model.save('DOGS-VS-CATS-CNN-MODEL.h5')

```

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the nat

```

▼ Evaluate the model

```

[19] loss , accuracy = model.evaluate(testing_set)
print(f"Accuracy : {accuracy:.4f}")

```

```

157/157 ━━━━━━━━━━━ 9s 59ms/step - accuracy: 0.8972 - loss: 0.2389
Accuracy : 0.8950

```

```

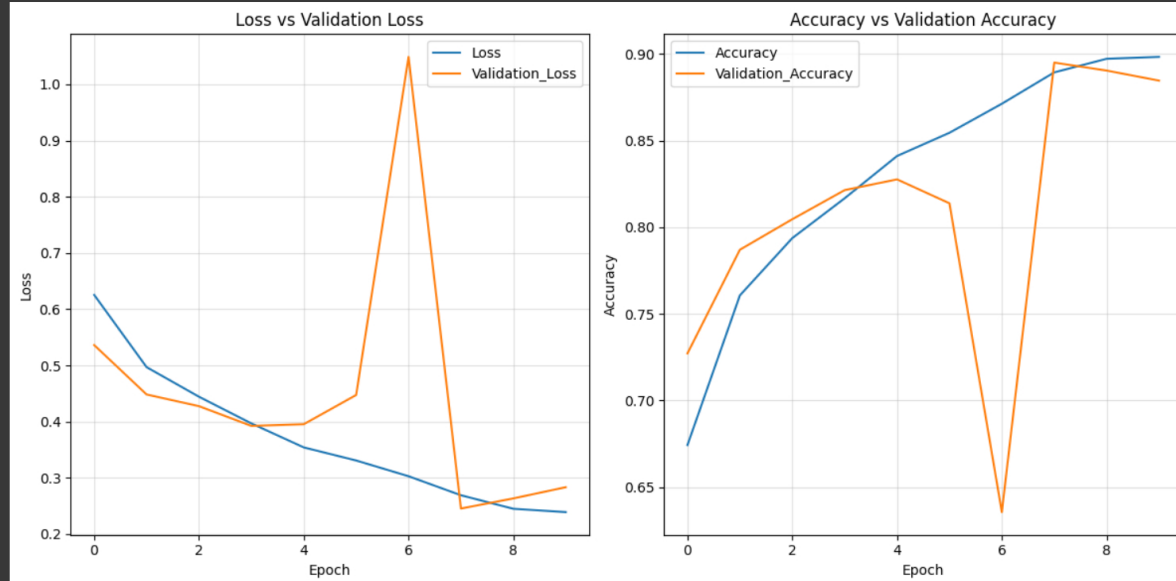
[20] plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.plot(history.history['loss'],label='loss')
plt.plot(history.history['val_loss'],label='Validation_Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Validation Loss")
plt.legend()
plt.grid(alpha=0.40)

plt.subplot(1,2,2)
plt.plot(history.history['accuracy'],label='Accuracy')
plt.plot(history.history['val_accuracy'],label='Validation_Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Validation Accuracy")
plt.legend()
plt.grid(alpha=0.40)

```

```
plt.tight_layout()
plt.show()
```

[]



Testing

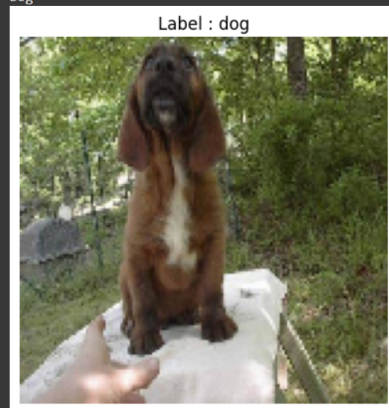
```
[21] def predict_image(image_path):
    img = load_img(image_path, target_size=(150, 150))
    plt.imshow(img)
    plt.title(f"Label : {image_path.split('/')[2][0:3]}")
    plt.axis('off')
    img = np.expand_dims(img, axis=0)

    result = model.predict(img)
    print("Dog" if result >= 0.5 else "Cat")

predict_image('dogs_vs_cats/test/dogs/dog.5.jpg')
```

[]

1/1 — 1s 758ms/step
Dog



```
[22] predict_image('dogs_vs_cats/test/cats/cat.83.jpg')
```

[]

1/1 — 0s 31ms/step
Cat



[22] Start coding or generate with AI.

✓ [22] Start coding or generate with AI.

Double-click (or enter) to edit

Colab paid products - Cancel contracts here

🔗 Variables 📄 Terminal

✓ 12:58 📄 T4 (Python 3)